EKS Session

Summary 23-04-2023

- **AWS IAM** is a service provided by AWS that enables you to manage access to AWS services. IAM allows you to create and manage AWS users, assign permissions to them, and control access to your AWS resources.

- **Role-based access control (RBAC)** is a method of restricting access to resources based on the roles assigned to users. With IAM, you can implement RBAC to control access to AWS resources for users within your AWS account.

- IAM is only used to manage access to AWS services and resources, and it does not provide RBAC for non-AWS services.

- IAM does not provide direct management of Kubernetes resources such as deployments, replicasets, and services.

- RBAC is a concept that can be implemented in various systems and applications, including AWS and Kubernetes.

- AWS provides its own implementation of RBAC through its IAM service. With AWS IAM, you can manage access to AWS services and resources.

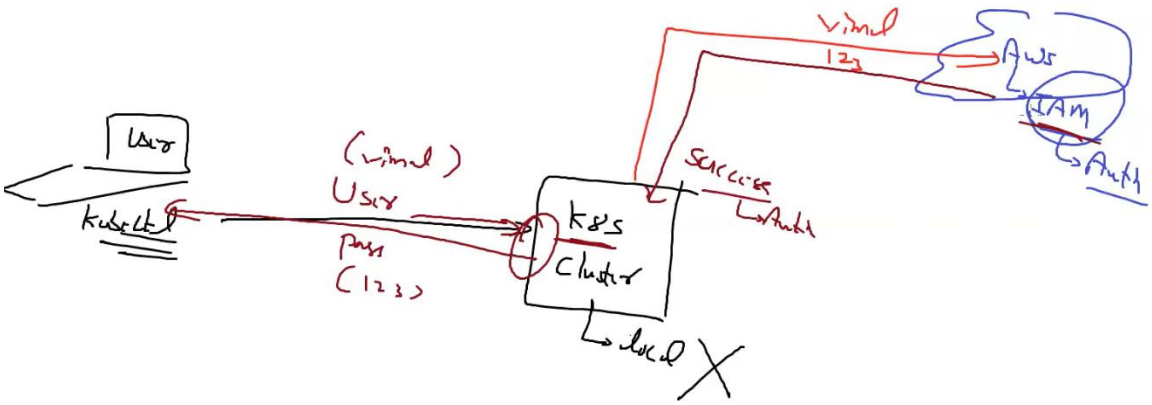
- Kubernetes also provides its own implementation of RBAC, which allows you to manage access to Kubernetes resources within a cluster.

- **Authentication and RBAC** are two essential components of access control systems.

- Authentication is the process of verifying the identity of a user. This process is usually achieved through a combination of a username and a password.

- **There are two main approaches to authenticating users in Kubernetes**: using an AWS IAM or using Kubernetes' built-in local authentication mechanisms.

- You can see how AWS IAM will be used for authentication in the example below.

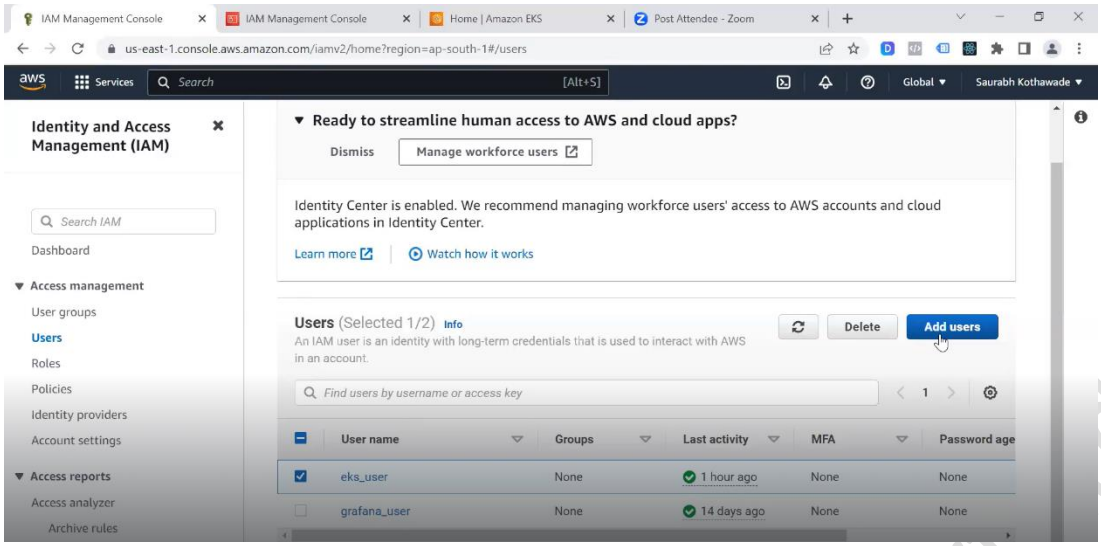


- **Single Sign-On (SSO)** is an authentication mechanism that allows users to access multiple applications or services with a single set of login credentials. In other words, users only need to authenticate once to access all the applications that they are authorized to use, instead of having to remember separate usernames and passwords for each application.

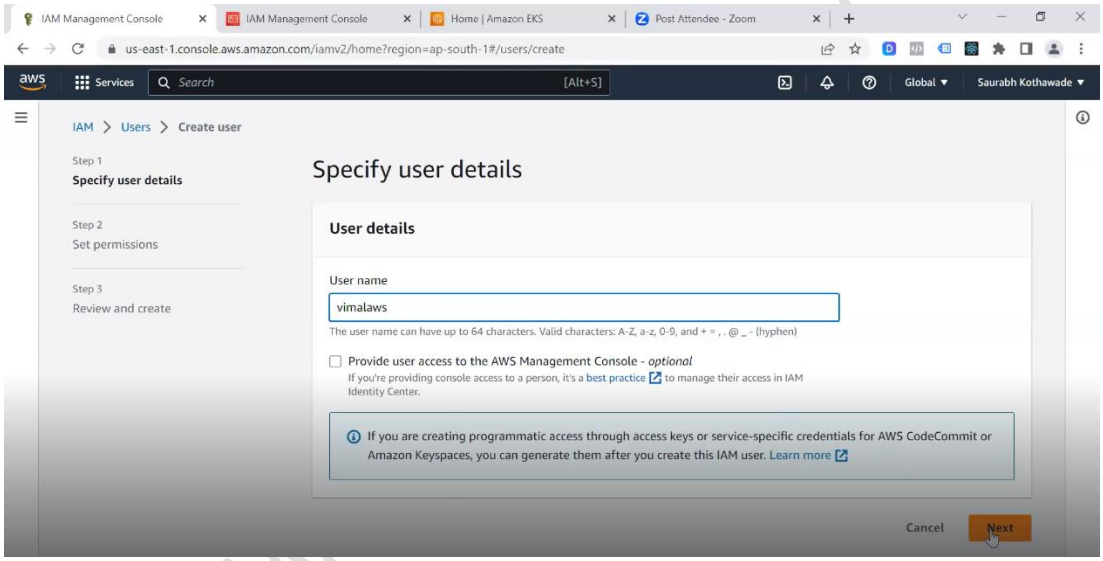
- If we talk about the example from above, both AWS and Kubernetes user authentication will be done via AWS IAM.

- **Create an IAM user:**

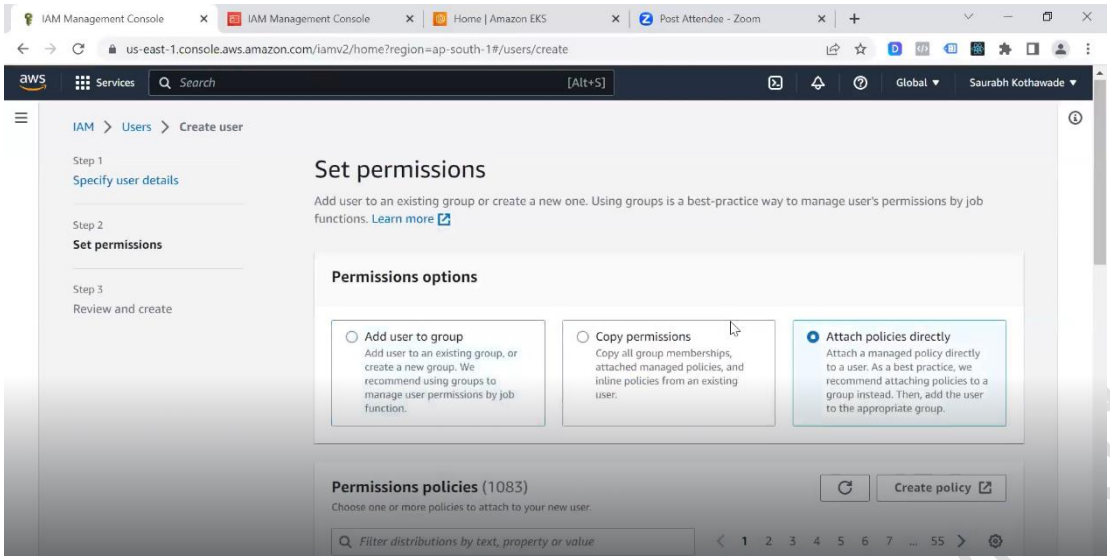
  - Click the "Add user" button to create a new IAM user.



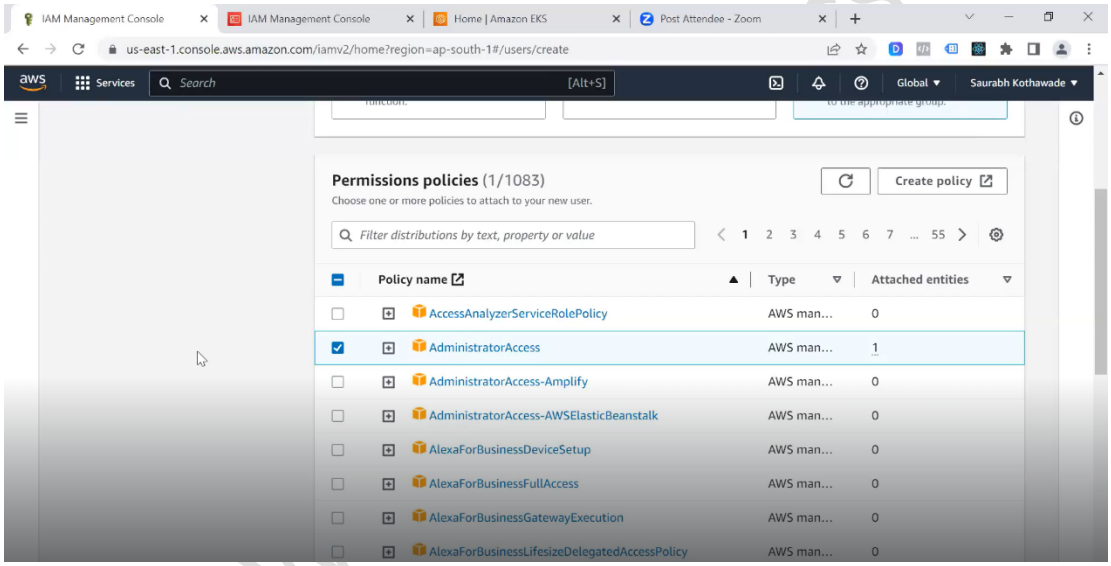
- Enter a user name for the new IAM user.



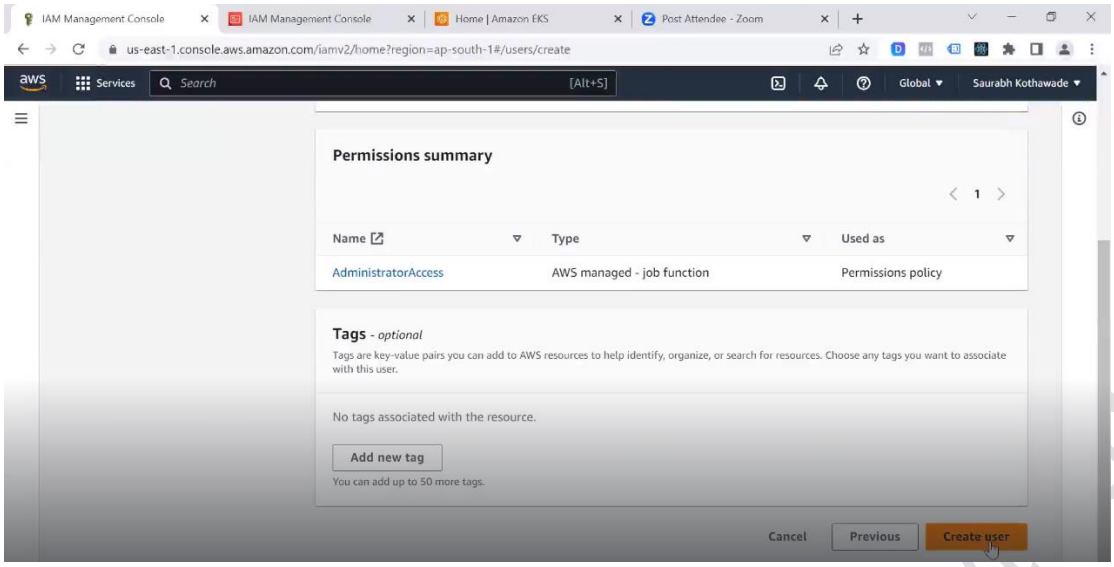
- On the "Set permissions" page, select "Attach policied directly".



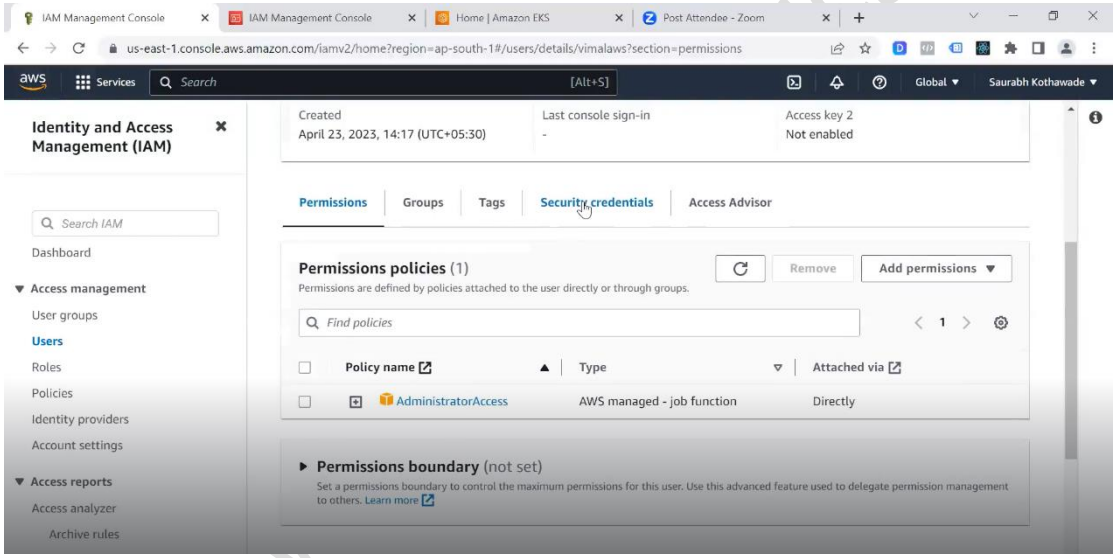
- Select the appropriate permissions for the user.



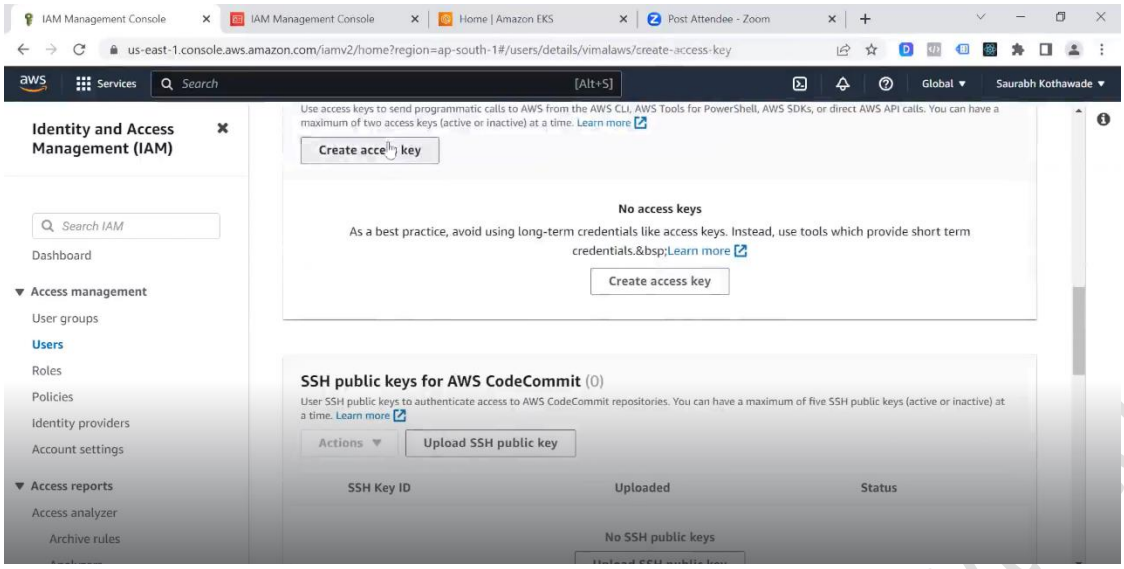
- Click on “Create user”.



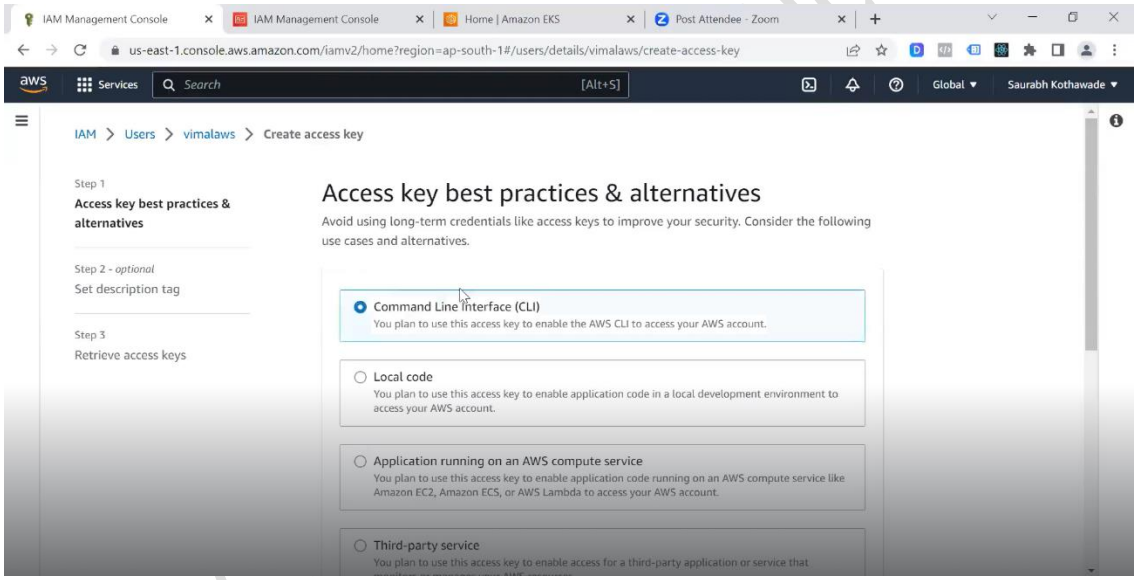
- Go to Security credentials.

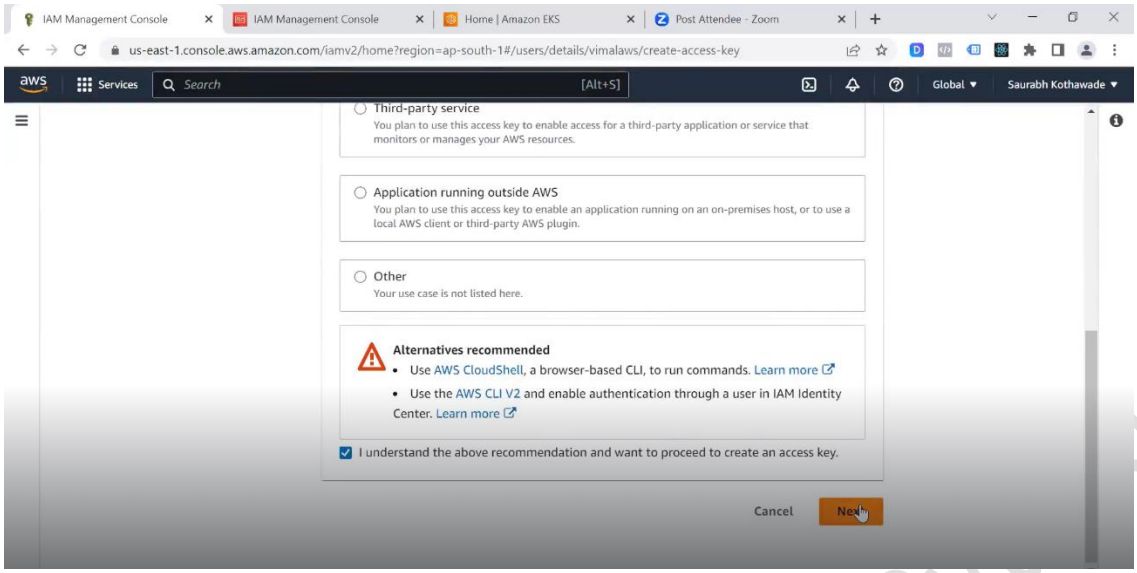


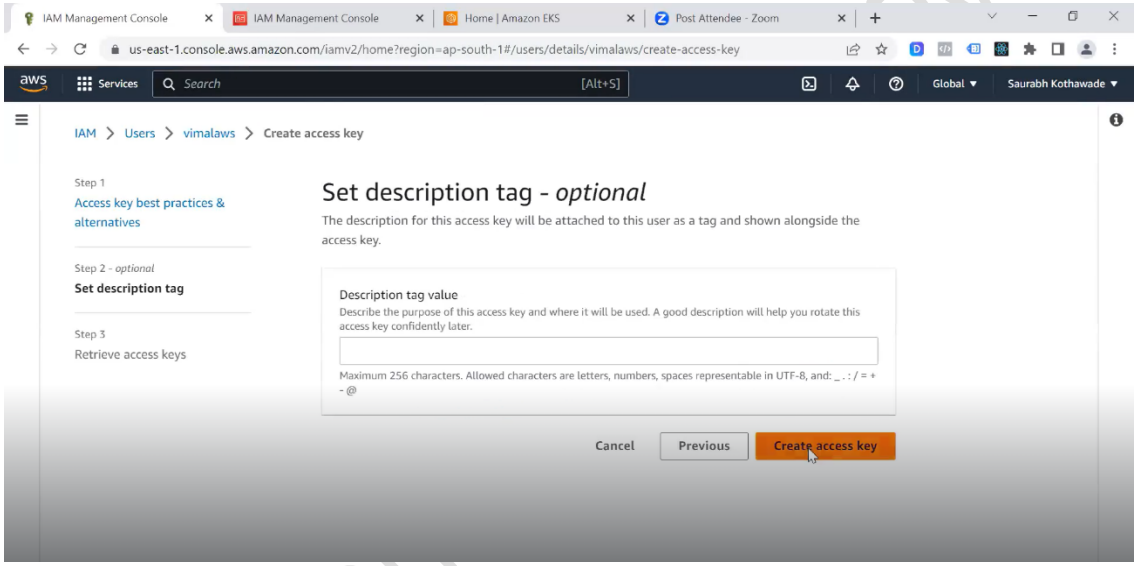
- Click on “Create access key”.



- Select “Command Line Interface(CLI)”.

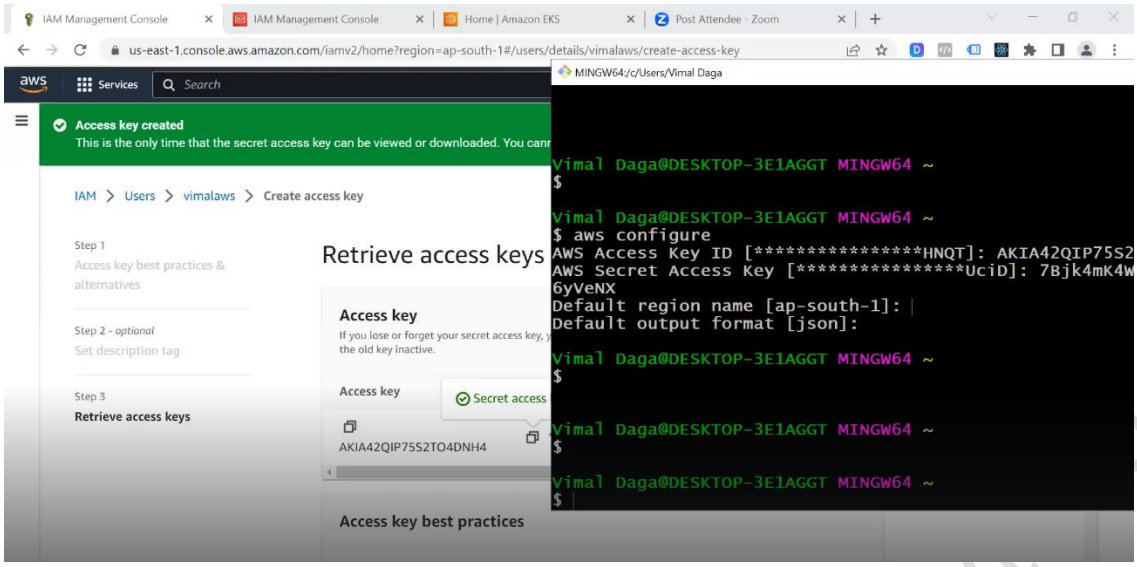






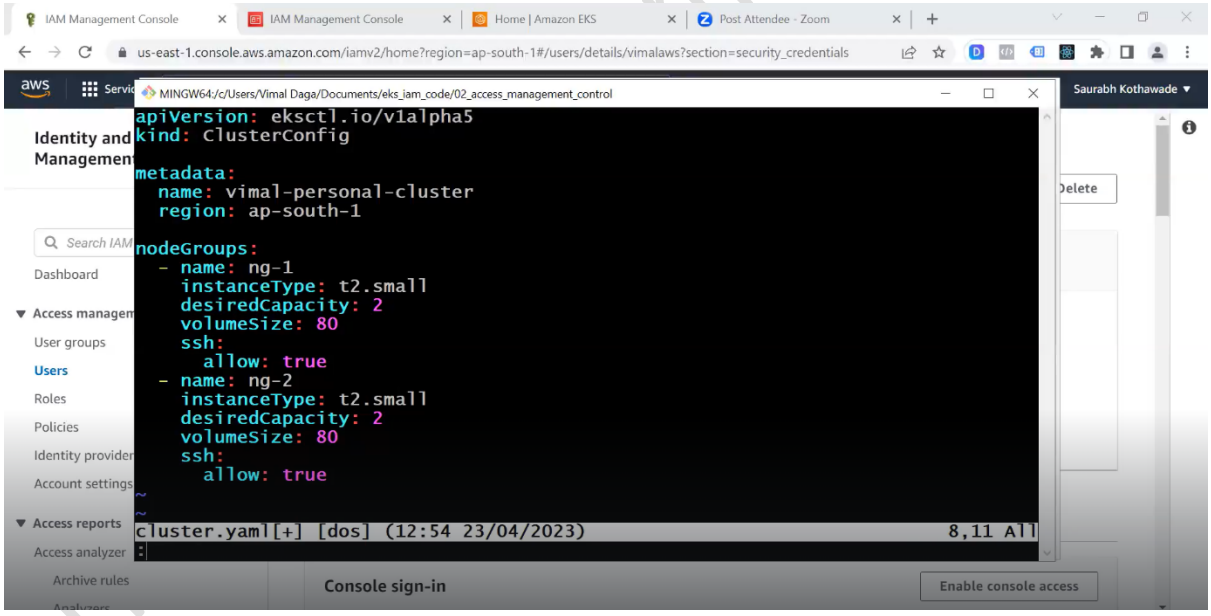
- Configure the AWS CLI (Command Line Interface) with your AWS account.

[EKS]



- **Create an EKS Cluster:**

 # vim cluster.yml



 # eksctl create cluster -f cluster.yml

- When you run the eksctl command, it first checks for AWS credentials.

- In Kubernetes, all resources are managed in **namespaces**. Each Kubernetes resource, such as pods, services, and deployments, is assigned to a namespace.

- In Kubernetes, you can use namespaces to apply RBAC policies to resources. For example, you might want that allows a user to create and delete pods in a specific namespace.

- A **role** is a set of permissions that defines a user's access to a specific set of resources within a namespace.

- For example, you might create a role that allows a user to create and delete pods in a specific namespace, and then create a **rolebinding** that assigns that role to the user.

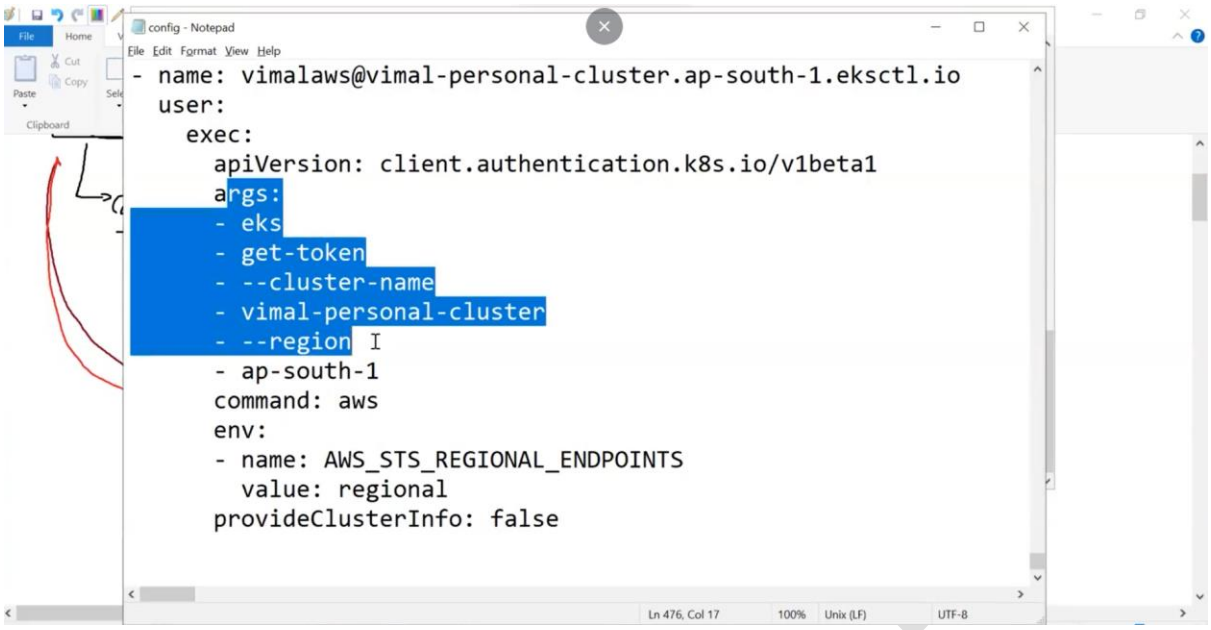
- Right now, we have three things in total: users, roles, and rolebinding.

  1) **Users will be created in AWS IAM**.
  2) **Roles and rolebinding will be created in Kubernetes only**.

- By default, the kubectl configuration file (kubeconfig) is updated to include the necessary credentials to authenticate with the EKS cluster using the admin kind of IAM role created during cluster creation. This allows you to immediately use kubectl to interact with the EKS cluster, without needing to manually configure credentials.

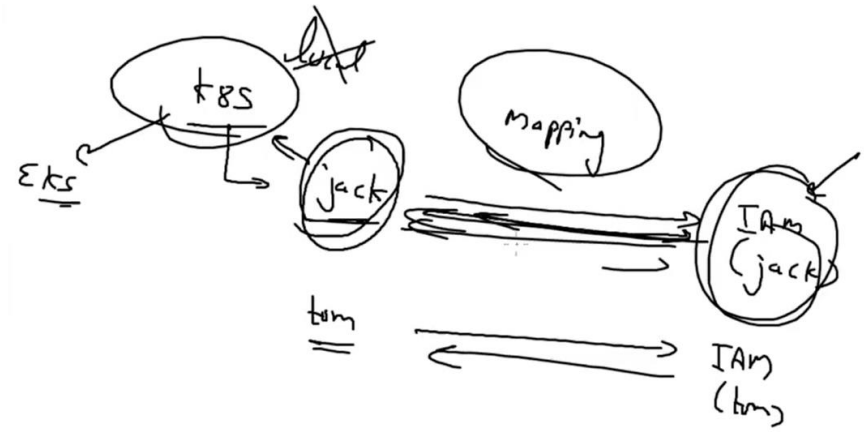
- When you run kubectl commands, it reads the kubeconfig file to determine how to connect to the Kubernetes cluster. The kubeconfig file is typically located in the .kube directory in your home directory, and it contains information such as the cluster endpoint, authentication credentials, and context information.

- When you run the kubectl command, it always read the kubeconfig file and internally executes the "*aws eks get-token --cluster-name vimal-cluster --region ap-south-1*" command for authentication.



- When configuring the AWS CLI, if we enter the incorrect access and secret keys, then the kubectl command to manage the K8s resources will fail because kubectl will internally only run the "aws eks" command.

- Since the user is outside of kubernetes, kubernetes is completely unaware of them. **User mapping** is the process of directing Kubernetes users to the locations of IAM users.

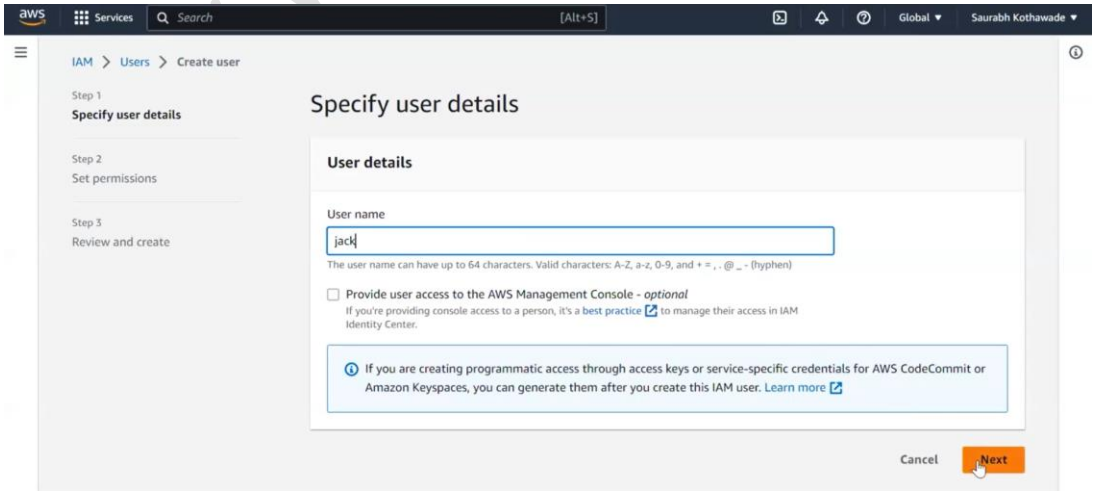


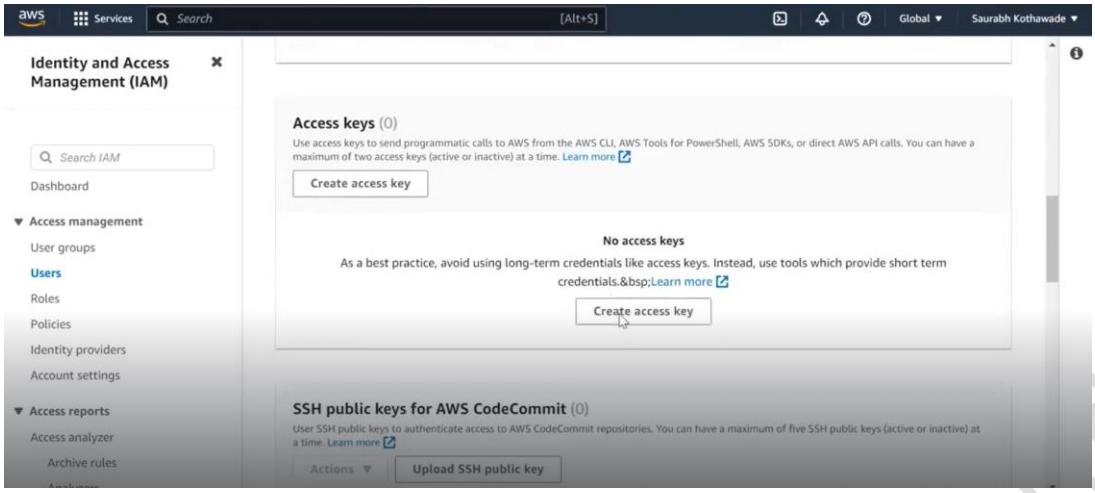
- The **aws-auth** ConfigMap is stored in the kube-system namespace of your Kubernetes cluster.

- The aws-auth ConfigMap is a Kubernetes ConfigMap that holds the configuration information.

```
$ kubectl  get cm -n |  kube-system
NAME                                 DATA   AGE
aws-auth                             2      25m
coredns                              1      37m
cp-vpc-resource-controller           0      37m
extension-apiserver-authentication   6      37m
kube-proxy                           1      37m
kube-proxy-config                    1      37m
kube-root-ca.crt                     1      37m
```

- Create a “jack” user with no power.



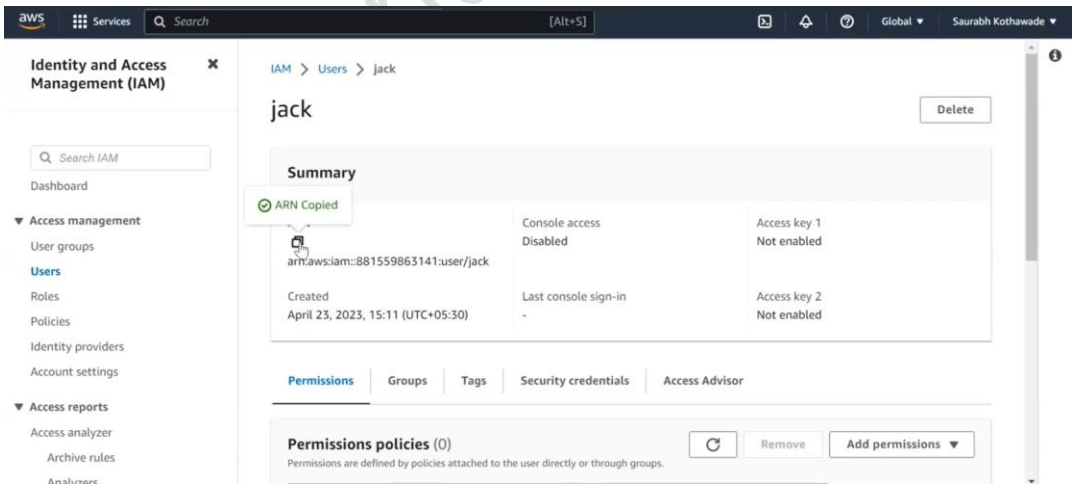


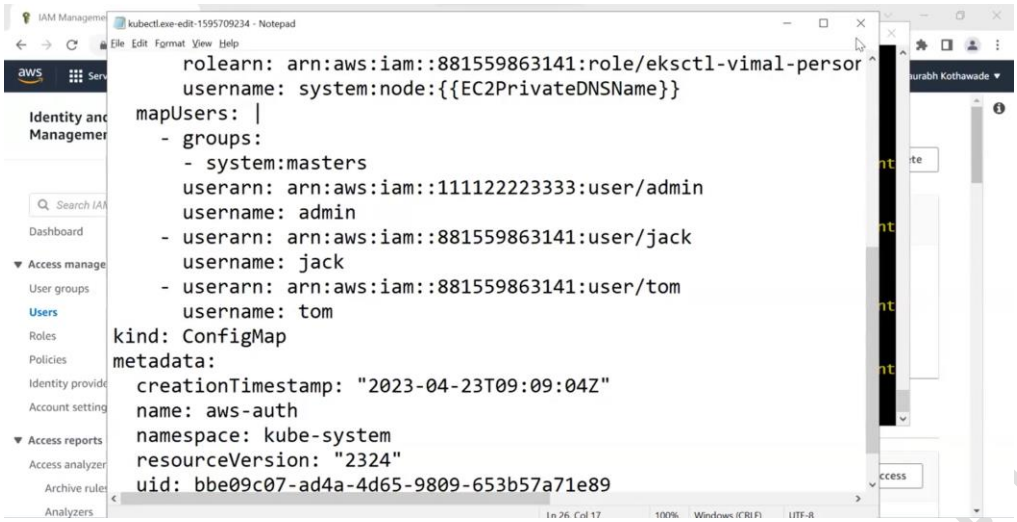
- In the aws-auth ConfigMap, you'll see a YAML file that includes a list of mapUsers sections.

- To add an IAM user, you would add a new entry to the mapUsers section, specifying the IAM user's ARN and the Kubernetes username that the IAM user should use when accessing the cluster.

- Edit the aws-auth ConfigMap

```
$ kubectl edit cm aws-auth -n kube-system
```





- Login to jack user: You will log in to Kubernetes as a jack user as soon as you log in to a jack user.

```
$ aws configure
AWS Access Key ID [****************V7N3]: AKIA42QIP75SVMKB2QAQ
AWS Secret Access Key [****************Gdo7]: nTkFO5ufwmuulA5TSRL/Gia2kvP9EKJ3OR
xGIdfx
Default region name [ap-south-1]:
Default output format [json]:
```

- We currently simply perform authentication; we haven't applied for RBAC, so we are unable to access any K8s resources.

```
$ kubectl get nodes
Error from server (Forbidden): nodes is forbidden: User "jack" cannot list resou
rce "nodes" in API group "" at the cluster scope

Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Documents/eks_iam_code/02_access_management
_control
$ kubectl get pods
Error from server (Forbidden): pods is forbidden: User "jack" cannot list resour
ce "pods" in API group "" in the namespace "default"
```

- The resources that we specify in the YAML file will be accessible when we create roles and rolebinding.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: my-vimal-custom-role-ro-1
rules:
  - apiGroups:
      - ""
    resources: [ "*" ]
    verbs:
      - get
      - list
      - watch
  - apiGroups:
      - extensions
    resources: [ "*" ]
    verbs:
      - get
      - list
      - watch
  - apiGroups:
      - apps
    resources: [ "*" ]
eks_trainee.yaml[+] [dos] (12:54 23/04/2023)                    4,12 Top
-- INSERT --
```

```
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: my-cluster-role-binding-jack
subjects:
  - kind: User
    name: jack
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: my-vimal-custom-role-ro-1
  apiGroup: rbac.authorization.k8s.io
```

```
$ kubectl apply -f eks_trainee.yaml
clusterrole.rbac.authorization.k8s.io/my-vimal-custom-role-ro-1 created
clusterrolebinding.rbac.authorization.k8s.io/my-cluster-role-binding-jack create
d
```

- View Kubernetes clusterrolebindings.

```
$ kubectl get clusterrolebinding  my-cluster-role-binding-jack
NAME                          ROLE                                   AGE
my-cluster-role-binding-jack   ClusterRole/my-vimal-custom-role-ro-1   60s
```

```
$ kubectl describe  clusterrolebinding  my-cluster-role-binding-jack
Name:         my-cluster-role-binding-jack
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  my-vimal-custom-role-ro-1
Subjects:
  Kind  Name  Namespace
  ----  ----  ---------
  User  jack
```

- Now once we have logged in as the Jack user, we can see that we have access to the K8s resources.

```
$ aws configure
AWS Access Key ID [****************R7VU]: AKIA42QIP75SVMKB2QAQ
AWS Secret Access Key [****************E7LY]: nTkFO5ufwmuulA5TSRL/Gia2kvP9EKJ3OR
xGIdfx
Default region name [ap-south-1]:
Default output format [json]:
```

```
$ kubectl get pods
No resources found in default namespace.

Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Documents/eks_iam_code/02_access_management
_control/role_mappings
$ kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.100.0.1   <none>        443/TCP   75m
```

```
$ kubectl create deployment myd --image=httpd
error: failed to create deployment: deployments.apps is forbidden: User "jack" c
annot create resource "deployments" in API group "apps" in the namespace "defaul
t"
```

- Create a role and bind that role with tom user.

```
apiVersion: rbac.authorization.k8s.io/
kind: ClusterRole
metadata:
  name: my-developer-clusterrole-tom-rw-1
rules:
  - apiGroups: [""]
    resources: ["nodes", "namespaces", "pods"]
    verbs: ["get", "list"]
  - apiGroups: ["apps"]
    resources: ["deployments" ,"daemonsets" ,"statefulsets" ,"replicasets"]
    verbs: ["get", "list", "create"]
  - apiGroups: [ "batch"]
    resources: ["jobs"]
    verbs: ["get", "list"]
```

[EKS]

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: my-role-developer-clusterrole-binging-tom-1
subjects:
  - kind: User
    name: tom
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: my-developer-clusterrole-tom-rw-1
  apiGroup: rbac.authorization.k8s.io
```

```
$ kubectl apply -f eks_developer.yaml
clusterrole.rbac.authorization.k8s.io/my-developer-clusterrole-tom-rw-1 created
clusterrolebinding.rbac.authorization.k8s.io/my-role-developer-clusterrole-bingi
ng-tom-1 created
```

```
$ kubectl get pods
No resources found in default namespace.

Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Documents/eks_iam_code/02_access_management
_control/role_mappings
$ kubectl get deploy
No resources found in default namespace.

Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Documents/eks_iam_code/02_access_management
_control/role_mappings
$ kubectl create deployment myd --image=httpd
deployment.apps/myd created
```